

Chapter 7. Modern Frameworks

Utilizing Virtual Threads

The best way to predict the future is to invent it.

—Alan Kay

With the introduction of virtual threads in JDK 21, the Java ecosystem has witnessed a significant shift in how frameworks handle concurrency. Many modern frameworks have embraced virtual threads to improve performance, scalability, and resource efficiency.

In this chapter, we will explore how leading frameworks, such as Spring Boot, Quarkus, and Jakarta EE, are integrating virtual threads. However, we will not go into the inner workings of these frameworks, as I assume those interested would prefer to explore each one in depth on their own.

Let's begin.

Spring Boot

[Spring Boot](#) is one of the de facto frameworks for building enterprise applications in the Java ecosystem.

Historically, Spring Boot web applications have primarily followed a *thread-per-request* model, where a dedicated platform thread handles each incoming client request. While this approach works well under moderate loads, it encounters scalability challenges when dealing with a high volume of concurrent I/O-bound requests, as these requests are always served through platform threads.

To mitigate this, Spring Boot introduced asynchronous programming capabilities with annotations like [@Async](#) and abstractions such as [TaskExecutor](#). However, this approach still relied on limited pools of platform threads, as shown in the following example:

```
@Configuration
public class ThreadPoolConfig {
    @Bean
    public AsyncTaskExecutor applicationTaskExecutor() {
```

```

        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(100);
        executor.initialize();
        return executor;
    }
}

```

Here, every request or task consumes a thread from a finite pool, potentially leading to performance bottlenecks under heavy loads.

With the release of Spring Boot 3.2, built upon Spring Framework 6.1, in November 2023, official support for Java 21, which finalized virtual threads, was introduced. While Java 17 remains the baseline JDK, Java 21 is now treated as a first-class runtime environment within the framework. A significant aspect of this support is the provision of a simple configuration property to enable virtual threads:

```

# application.properties
spring.threads.virtual.enabled=true

```

Or in YAML format:

```

# application.yml
spring:
  threads:
    virtual:
      enabled: true

```

Setting this configuration property enables virtual threads throughout the application. Spring Boot automatically configures an AsyncTaskExecutor backed by virtual threads (via a SimpleAsyncTaskExecutor) when no custom executor bean is defined. This configuration applies seamlessly across several concurrency-related features, such as:

- Asynchronous task execution (@EnableAsync)
- Asynchronous handling of Callable return values in Spring
- Spring Web MVC's asynchronous request processing
- Spring WebFlux's support for occasional blocking execution

Let's look at the following example:

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;

```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetingsController {
    private static final Logger LOGGER
        = LoggerFactory.getLogger(GreetingsController.class.getName());

    @GetMapping("/hello")
    public String hello() {
        LOGGER.info("Received request for /hello");
        LOGGER.info("Running on {}", Thread.currentThread());
        return "Hello from Spring Boot";
    }
}
```

While virtual threads are enabled, if we run the Spring Boot application and then send a curl request to the endpoint, we will see the following log in the console:

```
06:11:51.355 [tomcat-handler-0] INFO c.b.m.c.c.GreetingsController - Rece:
quest for /hello
06:11:51.356 [tomcat-handler-0] INFO c.b.m.c.c.GreetingsController - Runn:
VirtualThread[#63,tomcat-handler-0]/runnable@ForkJoinPool-1-worker-1
```

Consider a controller method that handles asynchronous requests:

```
import org.slf4j.Logger;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.concurrent.Callable;

@RestController
public class AsyncController {
    private static final Logger LOGGER
        = LoggerFactory.getLogger(AsyncController.class.getName());

    @GetMapping("/async-call")
    public Callable<String> handleAsyncRequest() {
        return () -> {
            Thread.sleep(500); // Simulate an I/O-bound operation
            LOGGER.info("Running on {}", Thread.currentThread());
            return "Hello from Virtual Thread!";
        };
    }
}
```

If you enable virtual threads, you will see output like this:

```
06:55:01.784 [task-1] INFO c.b.m.c.controller.AsyncController - Running on
```

```
alThread[#74,task-1]/runnable@ForkJoinPool-1-worker-1
```

Now, let's explore another practical scenario involving scheduled tasks:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@EnableScheduling
@Component
public class ScheduledTasks {
    private static final Logger LOGGER
        = LoggerFactory.getLogger(ScheduledTasks.class.getName());
    @Scheduled(fixedRate = 1000)
    public void scheduledTask() {
        LOGGER.info("Scheduled task running on: {}", Thread.currentThread().getName());
    }
}
```

With virtual threads enabled, your scheduled task logs will resemble this:

```
07:01:51.082 [scheduling-36] INFO c.b.m.c.controller.ScheduledTasks - Scheduled task running on: VirtualThread[#105,scheduling-36]/runnable@ForkJoinPool-1-worker-5
```

NOTE

Spring Boot conveniently auto-configures executors and schedulers for task execution. When virtual threads are enabled, it automatically uses [SimpleAsyncTaskExecutor](#) (named `applicationTaskExecutor`) and [SimpleAsyncTaskScheduler](#) (named `taskScheduler`). Without virtual threads, it defaults to a [ThreadPoolTaskExecutor](#) and [ThreadPoolTaskScheduler](#) with configurable settings. You can easily customize these defaults or create custom executors and schedulers using Spring Boot's provided builder classes. This seamless configuration helps simplify concurrency management and optimize performance in your applications.

Manual Configuration

If we're using an earlier version of Spring Boot (pre 3.2) or prefer more control over our executor configuration, we can explicitly define a virtual-thread-backed executor.

Let's consider how this customization (`applicationTaskExecutor`) works in practice:

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.task.AsyncTaskExecutor;
import org.springframework.core.task.support.TaskExecutorAdapter;
import java.util.concurrent.Executors;

@Configuration
public class VirtualThreadConfig {
    @Bean
    public AsyncTaskExecutor applicationTaskExecutor() {
        return new TaskExecutorAdapter(Executors.newVirtualThreadPerTaskExecutor());
    }
}

```

With this configuration, any use of `@Async` methods in your application will automatically run on virtual threads instead of traditional platform threads, as illustrated here:

```

import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;
import java.util.concurrent.CompletableFuture;

@Component
public class RemoteApiService {
    @Async
    public CompletableFuture<String> fetchDataFromRemoteApi() {
        try {
            Thread.sleep(1000); // Simulating an I/O-bound operation
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return CompletableFuture
            .completedFuture("Data fetched using virtual thread");
    }
}

```

In Spring's asynchronous programming model, methods annotated with `@Async` rely on the configured `applicationTaskExecutor`. If we replace the default executor with a virtual-thread-based executor, our `@Async` methods seamlessly run on lightweight virtual threads.

TIP

When configuring a virtual thread executor manually in Spring Boot, it's important to understand the distinction between using `TaskExecutorAdapter` and `Executors.newVirtualThreadPerTaskExecutor()`. While both enable virtual threads, `TaskExecutorAdapter` provides better integration with Spring's lifecycle management for beans. Consider Spring's management capabilities (e.g., initialization, shutdown) when choosing the appropriate approach.

Additionally, you can leverage virtual threads for your embedded web server to enhance scalability and resource utilization. For example, if you're using Apache Tomcat as your embedded web server, you can customize its protocol handler to utilize virtual threads by defining a `TomcatProtocolHandlerCustomizer` bean:

```
import org.springframework.boot.web.embedded.tomcat
    .TomcatProtocolHandlerCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.concurrent.Executors;

@Configuration
public class TomcatConfig {

    @Bean
    public TomcatProtocolHandlerCustomizer<?> virtualThreadExecutor() {
        return protocolHandler
            -> protocolHandler.setExecutor(
                Executors.newVirtualThreadPerTaskExecutor());
    }
}
```

Quarkus

Quarkus is a modern Java framework designed to emphasize developer productivity, rapid startup times, and efficient performance. It is ideal for building microservices and serverless applications. Built on Vert.x with a reactive programming paradigm, Quarkus has deliberately and explicitly integrated support for virtual threads, enhancing its suitability for concurrent applications.

Unlike a global approach, Quarkus introduces a targeted way to adopt virtual threads through specific annotations. Developers can selectively enable virtual threads on specific endpoints or services by annotating methods with `@RunOnVirtualThread`. This annotation instructs Quarkus to execute the annotated method on a virtual thread instead of a traditional

platform thread. The framework efficiently handles the lifecycle of these virtual threads, including creation, execution, and lifecycle management.

Quarkus integrates virtual threads mainly through its reactive extensions, combining reactive programming with virtual threading. This approach helps Quarkus applications efficiently handle concurrency, making them particularly suitable for microservices and serverless environments, thanks to their quick startup and efficient resource use.

NOTE

Quarkus leverages Vert.x as its underlying reactive engine. Vert.x's event-loop concurrency model is different from the traditional thread-per-request model. Quarkus's `@RunOnVirtualThread` annotation allows developers to offload specific tasks from the Vert.x event loop to virtual threads, preventing blocking operations from impacting the event loop's responsiveness. This combination of reactive programming and virtual threads enhances Quarkus's ability to efficiently handle concurrency.

Here's a practical example of how virtual threads are utilized in Quarkus:

```
package ca.bazlur;
import io.smallrye.common.annotation.RunOnVirtualThread;
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RestClient;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Path("/greetings")
public class VirtualThreadApp {
    private static final Logger logger
        = LoggerFactory.getLogger(VirtualThreadApp.class);

    @Inject
    @RestClient
    RemoteService remoteService;

    @GET
    @RunOnVirtualThread
    @Produces(MediaType.TEXT_PLAIN)
    public String process() {
        logger.info("Received greetings request");
        var response = remoteService.greetings();
        logger.info("Received response: {}", response);
        logger.info("Running on {}", Thread.currentThread());
        return response.toUpperCase();
    }
}
```

```

    }
}

```

The `process()` method, annotated with `@RunOnVirtualThread`, will be executed on a virtual thread.

If we run the code, we will get the following log in the console:

```

2025-03-16 00:12:31,453 INFO [ca.baz.VirtualThreadApp] (quarkus-virtual-tl
1) Received greetings request
2025-03-16 00:12:31,458 INFO [ca.baz.VirtualThreadApp] (quarkus-virtual-tl
1) Received response: Hey!
2025-03-16 00:12:31,458 INFO [ca.baz.VirtualThreadApp] (quarkus-virtual-tl
1) Running on VirtualThread[#219,quarkus-virtual-thread-1]/runnable@ForkJo
-1-worker-2

```

This log demonstrates that the `process` method is indeed running on virtual threads.

To enable this remote service, we have created an interface as follows:

```

import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;

@Path("/remote")
@RegisterRestClient
public interface RemoteService {
    @Path("/greetings")
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    String greetings();
}

```

To register this, we need the following item in the `application.properties` file:

```

quarkus.rest-client."ca.bazlur.RemoteService".url=http://localhost:8081

```

We have just created a simple REST endpoint for external services that serves random greeting messages. We can easily create this REST endpoint by using Java's built-in HTTP server capabilities. The JDK includes classes such as [HttpServer](#) and [HttpHandler](#), which allow us to

quickly set up lightweight HTTP services without additional frameworks or dependencies:

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpExchange;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.util.Random;
import java.util.concurrent.Executors;

public class SimpleHttpServer {
    private static final String[] GREETINGS = {
        "Hello, world!",
        "Hi there!",
        "Greetings!",
        "Good day!",
        "Hey!",
        "Howdy!",
        "Hola!",
        "Bonjour!",
        "Ciao!"
    };

    public static void main(String[] args) throws IOException {
        int port = 8081;
        HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
        server.createContext("/remote/greetings", new GreetingHandler());
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        System.out.println("Server started on port " + port);
        server.start();
    }

    static class GreetingHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            if ("GET".equalsIgnoreCase(exchange.getRequestMethod())) {
                String response = getRandomGreeting();
                exchange.sendResponseHeaders(200, response.length());
                try (OutputStream os = exchange.getResponseBody()) {
                    os.write(response.getBytes());
                }
            } else {
                exchange.sendResponseHeaders(405, -1); // Method Not Allowed
            }
        }

        private String getRandomGreeting() {
            Random random = new Random();
            return GREETINGS[random.nextInt(GREETINGS.length)];
        }
    }
}
```

```
}  
}
```

TIP

We can directly run this server using Java's source-file launching feature, available from Java 11 onward. This allows us to execute single-file Java programs without explicit compilation steps. Simply open your terminal and run the command:

```
java SimpleHttpServer.java
```

This command compiles and runs the file in one step, eliminating the need for manually using `javac`. It's ideal for quickly prototyping or running simple stand-alone Java applications.

We can even integrate Quarkus's reactive programming capabilities with virtual threads. Let's look at the following code:

```
import jakarta.inject.Inject;  
import jakarta.ws.rs.GET;  
import jakarta.ws.rs.Path;  
import jakarta.ws.rs.Produces;  
import jakarta.ws.rs.core.MediaType;  
import org.eclipse.microprofile.rest.client.inject.RestClient;  
import io.smallrye.common.annotation.RunOnVirtualThread;  
import io.smallrye.mutiny.Uni;  
import java.time.Duration;  
import jakarta.enterprise.context.ApplicationScoped;  
  
@Path("/reactive")  
public class ReactiveResource {  
    @Inject  
    HelloService helloService;  
    @GET  
    @Path("/hello")  
    @RunOnVirtualThread  
    @Produces(MediaType.TEXT_PLAIN)  
    public String hello() {  
        return helloService.getHello()  
            .await()  
            .atMost(Duration.ofSeconds(5));  
    }  
}  
  
@ApplicationScoped  
class HelloService {  
    @Inject  
    @RestClient
```

```

        ExternalService externalService;
        Uni<String> getHello() {
            return externalService.hello();
        }
    }

    @Path("/reactive")
    @RegisterRestClient
    interface ExternalService {
        @GET
        @Path("/hello")
        Uni<String> hello();
    }

```

The `hello()` method in `ReactiveResource` calls an external service using a reactive REST client (`ExternalService`). The `getHello()` method returns a `Uni<String>`, which represents a deferred computation. Within the `hello()` method, `uni.await().atMost(Duration.ofSeconds(5))` is used to block the virtual thread until the result is available or a timeout occurs.

Jakarta EE

[Jakarta Concurrency](#) is a standard specification that enables applications to utilize concurrency while maintaining the benefits of running within a [Jakarta EE](#) runtime. The release of [Jakarta Concurrency 3.1](#) introduced support for virtual threads, allowing applications to leverage Java 21's lightweight threading model.

Virtual threads can be enabled in Jakarta Concurrency 3.1 by specifying `virtual = true` in these annotations:

- [@ManagedExecutorDefinition](#)
- [@ManagedScheduledExecutorDefinition](#)
- [@ManagedThreadFactory](#)

When running on Java 21, virtual threads are used; however, if the application runs on Java 17, which doesn't support virtual threads, Jakarta EE runtimes automatically fall back to using platform threads.

The following example demonstrates how to configure and use virtual threads in Jakarta Concurrency 3.1:

```

import jakarta.enterprise.concurrent.ManagedExecutorDefinition;
import jakarta.enterprise.concurrent.ManagedExecutorService;
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;

```

```

import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

import java.util.concurrent.ExecutionException;

@ManagedExecutorDefinition(name = "java:module/concurrent/virtual-executor",
    qualifiers = WithVirtualThreads.class,
    virtual = true)
@Path("/virtualThreads")
public class VirtualThreadExampleService {

    @Inject
    @WithVirtualThreads
    ManagedExecutorService virtualManagedExecutor;

    @Inject
    GreetingService greetingService;

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String virtualThreads()
        throws InterruptedException, ExecutionException {
        return virtualManagedExecutor.submit(() -> {
            System.out.println("Received request on virtual thread: "
                + Thread.currentThread());
            return greetingService.getRandomGreeting();
        }).get();
    }
}

```

Now, if we run the preceding code using the Open Liberty runtime we will hit the following endpoint:

```
GET http://localhost:9080/api/virtualThreads
```

You will see the part that executes to get the random greetings; it will be on the virtual threads:

```

[INFO] Received request on virtual thread: VirtualThread[#115,application[
ee-vthrads]/module[jakartaee-vthrads.war]/managedExecutorService[java:modu
urrent/virtual-executor]/concurrencyPolicy:1]/runnable@ForkJoinPool-1-work

```

Similarly, if we want to run virtual threads on a `ManagedScheduledExecutorService` that allows periodic or delayed task execution, we would do the following:

```

import jakarta.annotation.Resource;
import jakarta.enterprise.context.ApplicationScoped;

```

```

import jakarta.enterprise.concurrent.ManagedScheduledExecutorService;
import jakarta.enterprise.concurrent.ManagedScheduledExecutorDefinition;

import java.util.concurrent.TimeUnit;

@ApplicationScoped
@ManagedScheduledExecutorDefinition(
    name = "java:module/concurrent/virtual-scheduler",
    virtual = true // Enables Virtual Threads for Scheduled Tasks
)
public class VirtualThreadSchedulerExample {

    @Resource(lookup = "java:module/concurrent/virtual-scheduler")
    private ManagedScheduledExecutorService scheduledExecutor;

    public void scheduleTask() {
        scheduledExecutor.schedule(() -> {
            System.out.println("Scheduled task running in virtual thread: "
                + Thread.currentThread());
        }, 5, TimeUnit.SECONDS); // Delay execution by 5 seconds
    }
}

```

NOTE

While Jakarta Concurrency 3.1 provides the specifications for using virtual threads, the actual behavior can vary between Jakarta EE runtimes. You should be aware of potential compatibility issues and read the documentation of their specific Jakarta EE runtime (e.g., Open Liberty, Payara, WildFly) for implementation-specific details and best practices. For example, features and configurations might behave differently or might not be present in all Jakarta EE runtimes.

As of the writing of this book, Open Liberty 24.0.0.6-beta is the first Jakarta EE runtime to fully support Jakarta Concurrency 3.1 with virtual threads. However, as Jakarta EE 11 approaches its final release, other Jakarta EE runtimes, such as Payara, WildFly, TomEE, and GlassFish, are expected to incorporate virtual thread support.

In Closing

Besides the major frameworks like Jakarta EE, Spring, and Quarkus, several other frameworks have also embraced virtual threads to enhance concurrency efficiency in modern Java applications.

One such framework is [Helidon Nima](#), a microservice framework designed from the ground up to leverage virtual threads natively. Unlike traditional frameworks that retrofit virtual thread support, Helidon Nima is

built around virtual threads, making it a lightweight and high-performance solution for microservices.

Similarly, the [Micronaut](#) framework automatically utilizes virtual threads if the application runs on a JDK version that supports virtual threads (Java 21 or later). This means developers using Micronaut can transparently benefit from the performance improvements of virtual threads without additional configuration.

As Java continues to evolve, virtual threads have become a key feature for building scalable, efficient, and developer-friendly applications. Whether using Jakarta EE 11, Quarkus, Spring Boot 3, Helidon Níma, or Micronaut, developers now have powerful tools to write highly concurrent applications while maintaining a simple, imperative coding style.

With virtual threads now an integral part of the Java ecosystem, the future of Java concurrency looks more efficient, scalable, and developer-friendly than ever before.